MAE162
CAPSTONE I/II

Lab: Vision Node

Version 0.1.0

162 TAs and Maclab members

May 5, 2026

Warning!

- No food and drinks are allowed in the lab.
- Be careful with the hardware especially the electronics.
- Do not run the robot on the table — it may fall and be damaged.
- Be gentle to the buttons! You are liable for any damage made to the robot.
- Wash your hands after each experiment, some components may contain lead.

Contents

1 Overview **3**

2 Lab Activities **4**

2.1 Task 0: Update and Rebuild the Docker Image 4
2.2 Task 1: Set Up the Camera Service 5
2.3 Task 2: Verify Detections in Debug Mode 6
2.4 Task 3: Traffic Light LED Demo 7
2.5 Task 4: Drive Forward on Green Light 9
2.6 Task 5 (Bonus): Stop on Stop Sign 9

3 Lab Deliverables **10**

4 Technical Reference **11**

4.1 Vision Pipeline 11
4.2 Detection Topic and Message Format 12
4.3 Robot API Reference 12
4.4 Customising the Class Filter 13
4.5 Yellow Block Detection 14

1 Overview

The `vision` ROS2 package reads frames from the Raspberry Pi camera, runs YOLO object detection, and publishes results on the `/vision/detections` topic. Your robot node subscribes to this topic through the Robot API and can react to whatever the camera sees.

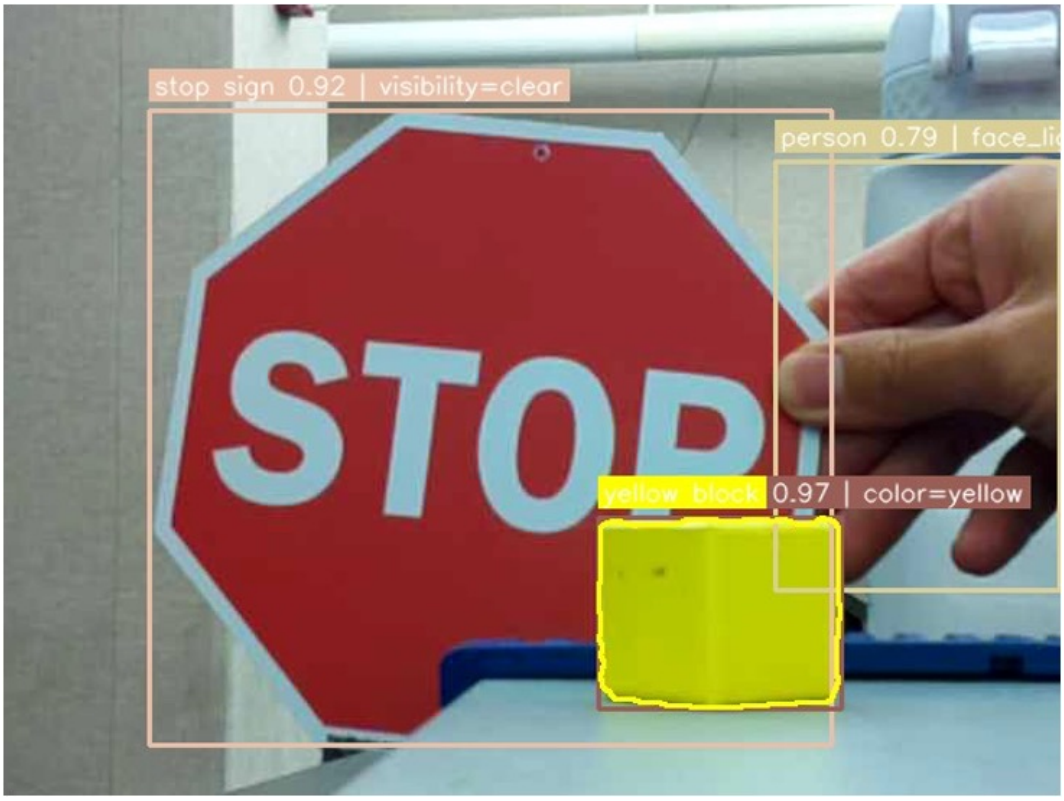

Figure 1: Vision node output: YOLO detections overlaid on a camera frame.

By the end of this lab you will have completed three deliverables, each building on the previous one:

1. **Traffic light LED demo** — The robot's onboard LEDs mirror the colour of a traffic light held in front of the camera.
2. **Drive on green** — The robot also drives forward when a green light is detected and stops otherwise.
3. **(Bonus) Stop on stop sign** — Additionally, the robot stops and holds whenever a stop sign is detected, regardless of the traffic light colour.

Estimated time: 60 minutes for Tasks 1–4; Task 0 and Bonus Task 5 are extra.

2 Lab Activities

2.1 Task 0: Update and Rebuild the Docker Image

Where to run: **RPi** Raspberry Pi (SSH terminal) | Only needed if your repository is not up to date. Skip this task if the TA confirms your image is current.

If the Docker image on your Pi was built from an older version of the repository, the vision node or bridge may be missing recent changes. Before pulling, make sure your fork has been merged with the latest upstream changes — you can do this from the GitHub web interface or in the terminal. Then pull the latest code and rebuild.

0.1 Pull Latest Changes **RPi**

From the repository root:

```
git pull
```

0.2 Rebuild the Docker Image **RPi**

```
COMPOSE=ros2_ws/docker/docker-compose.rpi.yml
docker compose -f $COMPOSE build
```

This may take several minutes on the first rebuild.

0.3 Restart the Container **RPi**

```
docker compose -f $COMPOSE down
docker compose -f $COMPOSE up -d
```

Confirm the container is running:

```
docker compose -f $COMPOSE ps
```

The `ros2_runtime` service should show `running`.

2.2 Task 1: Set Up the Camera Service

Where to run: **RPi** Raspberry Pi (SSH terminal) | One-time setup. If your team has already done this on this Pi, run only the check in step 1.2 to confirm the service is healthy.

The Raspberry Pi CSI camera runs on the *native* Ubuntu host (not inside Docker). A small systemd service captures frames with `rpicam-vid`, pipes them through `ffmpeg`, and writes them into a virtual V4L2 device at `/dev/video10`. The Docker container reads this virtual device exactly as if it were a USB webcam.

1.1 Install the Camera Service **RPi**

From the repository root on the Raspberry Pi:

```
./ros2_ws/host_camera/install.sh
```

The script installs `rpicam-apps`, `ffmpeg`, `v4l2loopback`, and a systemd service that starts automatically at boot. It will prompt for `sudo` if needed.

1.2 Verify the Camera **RPi**

```
./ros2_ws/host_camera/check.sh
```

Expected final line:

```
Results: 9 passed, 0 failed
```

The check script saves two test images you can inspect:

```
/tmp/pi_camera_check/host_loopback.jpg
/tmp/pi_camera_check/docker_loopback.jpg
```

If the check fails, inspect the service log:

```
sudo systemctl status pi-camera-feed.service --no-pager
sudo journalctl -u pi-camera-feed.service -n 40 --no-pager
```

A quick restart often resolves transient failures:

```
sudo systemctl restart pi-camera-feed.service
sleep 3
systemctl is-active pi-camera-feed.service   # should print: active
```

2.3 Task 2: Verify Detections in Debug Mode

Where to run: **RPi** Raspberry Pi (Docker shell) | The Docker container must be running. Enter it the same way as in the previous lab.

In debug mode the vision node saves an annotated camera frame to disk every second. This lets you confirm that the camera feed is working and that objects are being detected before writing any robot code.

2.1 Open a Shell Inside the Container **RPi**

```
COMPOSE=ros2_ws/docker/docker-compose.rpi.yml
docker compose -f $COMPOSE exec ros2_runtime bash
source /opt/ros/jazzy/setup.bash
source /ros2_ws/install/setup.bash
```

2.2 Launch the Vision Node in Debug Mode RPi

```
ros2 launch vision vision_debug.launch.py
```

You should see log lines similar to:

```
[vision_node]: Vision frame 640x480 total=240.3ms ncnn=228.1ms yolo=2
↪  yellow_block=0 total=2
```

Leave this terminal running.

2.3 View the Debug Image RPi

Debug images are saved inside the container at `/runtime_output/vision/latest.jpg`, which is mounted to the Raspberry Pi host at:

```
ros2_ws/runtime_output/vision/latest.jpg
```

Open this path in **VS Code's file browser** (via the Remote SSH extension). Click the file to preview it. The image refreshes on disk every second while the vision node is running — click away and back, or use the VS Code refresh button, to see the latest frame.

Hold a **traffic light** or **stop sign** in front of the camera and confirm that coloured bounding boxes appear around the detected objects. If no detections appear, try improving the lighting or moving the object closer.

2.4 Stop the Node

Press **Ctrl+C** in the terminal running the launch file when you are done.

2.4 Task 3: Traffic Light LED Demo

Where to run: RPi Raspberry Pi (two Docker shells) | You will need two terminal windows open inside the Docker container simultaneously. Open a second terminal and run the same `source` commands from Task 2.1 before continuing.

3.1 Start the Vision Node — Terminal A RPi

In the **first** Docker shell, start the vision node in production mode (no debug image saving, lower CPU overhead):

```
ros2 launch vision vision_production.launch.py
```

Keep this running throughout the rest of the lab.

3.2 Copy and Run the Example — Terminal B RPi

In the **second** Docker shell, navigate to the robot package and copy the example:

```
cd /ros2_ws/src/robot/robot/
cp examples/traffic_light_leds.py main.py
```

Make sure the bridge node is running (as in previous labs), then start the robot node:

```
ros2 run robot robot
```

3.3 Test the Demo RPi

Hold a red or green traffic light image (or a real light) in front of the camera:

- A **green** detection → the green LED turns on.
- A **red** detection → the red LED turns on.
- No fresh detection for 2 seconds → all LEDs turn off.

When this works, you have completed **Deliverable (i)**.

Take a moment to read `traffic_light_leds.py`. It is short and well-commented; Tasks 4 and 5 ask you to extend it.

2.5 Task 4: Drive Forward on Green Light

Where to run: **RPi** Raspberry Pi (Docker shell) | Stop the robot node from Task 3 before starting. The vision node in Terminal A should keep running.

Extend `main.py` so the robot drives forward whenever a green light is detected and stops otherwise. The Robot API reference in Section 4.3 lists the methods you will need.

Your implementation must satisfy the following requirements:

1. Call `robot.set_odometry_parameters()` in `configure_robot()` with the wheel geometry constants from `hardware_map` (`WHEEL_DIAMETER`, `WHEEL_BASE`, `INITIAL_THETA_DEG`, `LEFT_WHEEL_MOTOR`, `RIGHT_WHEEL_MOTOR`, and the corresponding `DIR_INVERTED` flags).
2. When a **green** traffic light is detected: drive forward at a constant speed using `robot.set_velocity()`, and keep the green LED on.
3. When a **red** traffic light is detected, or when no detection has arrived recently: stop the robot with `robot.stop()`.

Run the robot node and verify:

- Show a **green** light → robot drives forward, green LED on.
- Show a **red** light → robot stops, red LED on.
- Cover the camera → robot stops, LEDs off after 2 s.

When this works, you have completed **Deliverable (ii)**.

2.6 Task 5 (Bonus): Stop on Stop Sign

Where to run: **RPi** Raspberry Pi (Docker shell) | Continue editing `main.py` from Task 4.

Add stop-sign awareness so that a stop sign overrides any traffic light command: the robot stops and the red LED blinks whenever a stop sign is in view.

5.1 Extend the FSM RPi

At the very beginning of the `elif state == "WATCHING":` block, add a stop-sign check *before* the traffic light logic:

```
elif state == "WATCHING":
    now = time.monotonic()

    # Stop sign takes priority over traffic light colour.
    if robot.get_detections("stop sign"):
        robot.stop()
        robot.set_led(LED.RED, LED_BRIGHTNESS)
        robot.set_led(LED.GREEN, 0)
        last_shown_color = None
        lights_off_at = 0.0

    else:
       # --- existing traffic light + drive logic from Task 4 goes here ---
        traffic_light_color = find_traffic_light_color(robot)
        ...
```

`robot.get_detections("stop sign")` returns a list of all current stop-sign detections. It is non-empty whenever at least one stop sign is visible to the camera.

5.2 Test RPi

- Show a **green** light alone → robot drives forward.
- Show a **stop sign** (regardless of traffic light) → robot stops, red LED on.
- Remove the stop sign, show green again → robot resumes driving.

When this works, you have completed **Deliverable (iii)**.

3 Lab Deliverables

Demonstrate each deliverable to a TA in order.

1. **Traffic light LED demo** — Hold a red or green traffic light in front of the camera. The matching LED on the robot turns on; LEDs turn off when the light is removed for 2 seconds.
2. **Drive on green** — Show a green light: the robot drives forward. Show red or cover the camera: the robot stops. Demonstrate both transitions cleanly.
3. **(Bonus) Stop on stop sign** — While the robot is driving on green, introduce a stop sign. The robot stops immediately. Remove the stop sign and show green again: the robot resumes.

4 Technical Reference

This section is optional reading. Use it when you want to go deeper or customise the vision pipeline beyond what the tasks above cover.

4.1 Vision Pipeline

`Pi CSI camera` → `rpicam-vid + ffmpeg` → `/dev/video10` (v4l2loopback) → `vision_node` (Docker) → `/vision/detections` (ROS2 topic) → `robot node`

Key source files inside the `vision` package:

- `vision_node.py` — Main node: opens the camera, runs YOLO inference at 4 Hz, calls post-processors, and publishes detections.
- `model_utils.py` — YOLO NCNN inference wrapper and `DetectedObject` data class.
- `rule_based_detection.py` — Yellow block detector (HSV colour thresholding, always active).
- `traffic_light.py` — Classifies a detected traffic light crop into `red`, `green`, or `unknown`.
- `stop_sign.py` — Classifies stop sign visibility.
- `launch/vision_debug.launch.py` — Debug mode: saves annotated frames to `/runtime_output/vision/latest.jpg` at 1 Hz.

- `launch/vision_production.launch.py` — Production mode: no frame saving, lower CPU load.

4.2 Detection Topic and Message Format

The vision node publishes on the topic `/vision/detections` using the message type `bridge_interfaces/msg/VisionDetectionArray`.

Each detection in the array has these fields:

Field	Type	Meaning
`class_name`	string	Detected class, e.g. `"traffic light"`
`confidence`	float	Detection score, 0–1
x, y	int	Top-left corner of bounding box (pixels)
`width`, `height`	int	Bounding box size (pixels)
`attribute_names`	string[]	Attribute labels, e.g. `["color"]`
`attribute_values`	string[]	Attribute values, e.g. `["green"]`
`attribute_scores`	float[]	Confidence per attribute

You can inspect the raw topic from a Docker shell:

```
ros2 topic echo /vision/detections
```

4.3 Robot API Reference

Call `robot.enable_vision()` in `configure_robot()` before using any of the methods below.

Method	What it returns / does
`enable_vision()`	Subscribe to `/vision/detections`.
`is_vision_active(t)`	`True` if a message arrived within `t` seconds.
`get_detections(class_name)`	List of detection dicts for the given class (or all if omitted).
`get_detection_attribute(cls, attr)`	`(value, score)` for the first match, or `None`.
`set_velocity(linear, angular_deg_s)`	Body-frame velocity: mm/s forward, deg/s rotation.
`stop()`	Zero velocity on both drive wheels.

Each detection dict returned by `get_detections()` has the keys `class_name`, `confidence`, `x`, `y`, `width`, `height`, and `attributes`. `attributes` is itself a dict mapping attribute name to `{"value": ..., "score": ...}`.

4.4 Customising the Class Filter

By default the vision node detects **traffic lights**, **stop signs**, and **persons**. To add or remove COCO classes, edit the `CLASSES_OF_INTEREST` list at the top of `vision_node.py`:

```
CLASSES_OF_INTEREST = [
    "traffic light",
    "stop sign",
    "person",
    # "car",
    # "bus",
]
```

After editing, rebuild and restart the vision node. Valid class names are in the model metadata file: `vision/data/yolo26n_ncnn_imgsz_416/metadata.yaml`.

4.5 Yellow Block Detection

The vision node always runs an HSV-based yellow block detector alongside YOLO. If you call `robot.get_detections()` with no argument you will see `"yellow block"` entries in the list alongside YOLO results.

Calling `robot.get_detections("traffic light")` or any other specific class name already filters these out — you will not see yellow block detections unless you explicitly ask for them.

If you want to *completely* disable yellow block detection (e.g. to reduce CPU load when it is not needed), comment out the yellow block call in `vision_node.py`:

```
# In VisionNode.run():
yellow_block_detections, yellow_block_overlays =
↪  self._detect_yellow_block(frame)
# Comment the line above and replace with:
# yellow_block_detections, yellow_block_overlays = [], []
```